



Final PRD



Course

vibe project idea

Overview:

a minimalist, mobile-first web application designed for friends to effortlessly track and settle shared expenses through a sleek, modern interface. It features a unique dual-confirmation system and "Smart Netting" logic that automatically calculates algebraic balances, allowing users to settle multiple debts with a single click.

Core Features

1. Intelligent Dashboard & Financial Overview

- Real-time display of "Total to Pay" and "Total to Receive" balance cards
- Priority-based friends list displaying top pending amounts
- Quick-action floating button for instantaneous request creation
- High-level visibility of financial health at a glance

2. Relationship-Based Transaction Ledger

- Segregated views for "Current" (Active) and "Past" (Settled) transactions
- Individual friend profiles mapping full transaction histories
- Detailed breakdown of specific debts within the friendship context
- Automated archiving of requests once settlement is reached

3. Smart Request Engine

- Automated metadata capture (date and time)

- Multi-friend tagging for split-bill scenarios. Synchronized real-time status updates across both user accounts
- Role assignment via "I Paid" or "I Received" toggles
- Dynamic title and amount input with a minimalist UI

4. Algebraic "Smart Netting" System

- Automatic calculation of the net difference between two users
- Batch grouping of all pending requests into a single "Master Sum"
- Simplified settlement UI that replaces multiple debts with one final balance
- Streamlined resolution to reduce the number of individual payments needed

5. Dual-Confirmation Settlement Protocol

- Two-step verification requiring both "Paid" and "Received" confirmations
- Real-time status updates reflecting the "Pending Verification" state
- "Vanishing" logic that clears active requests only upon mutual agreement
- Prevention of unilateral debt clearance to maintain trust and transparency

6. Minimalist Mobile-First UX

- Responsive design optimized for mobile usage
- Aesthetic modern UI with high-contrast typography and soft color palettes
- Intuitive navigation patterns and gesture-friendly interactions
- Lightweight architecture for fast loading on mobile networks

User Experience

Design Philosophy

- **The "Glanceable" Dashboard:** The home screen follows a 2-second rule—users should understand exactly how much they owe and are owed within

two seconds of opening the app.

- **The Digital Handshake:** UX centers around the psychological satisfaction of "clearing" a debt.
The dual-confirmation (Paid/Received) acts as a digital handshake, fostering trust and transparency.
- **Frictionless Settlements:** The "Smart Netting" feature eliminates the math fatigue of multiple small transactions, reducing the user journey from calculating a sum to a single tap.
- **Haptic Minimalism:** Using a high-contrast, clutter-free interface to ensure that primary actions (Create, Pay, Confirm) are always within thumb's reach on mobile devices.
- Clean, minimalistic interface

User Personas

1. The Social Catalyst (The "Frontman")

- **Profile:** Often the one who swipes their card for group dinners, concert tickets, or shared rides.
- **Pain Point:** Keeping track of five different friends owing small, varying amounts and feeling awkward about "reminding" them.
- **User Goal:** To have a centralized list where they can see who has confirmed a payment and who is still pending, without manual record-keeping.

2. The High-Frequency Roommate

- **Profile:** Shares daily expenses like groceries, utilities, and household supplies with the same person.
- **Pain Point:** Sending \$5 back and forth multiple times a week via traditional banking apps, leading to "transaction clutter."
- **User Goal:** To use the **Algebraic Sum** feature to let expenses accumulate and settle the "Net Difference" once at the end of the month with a single transaction.

3. The Forgetful Payback-er

- **Profile:** A well-intentioned friend who often forgets they owe money until they check their messages.
- **Pain Point:** Losing track of "who owes what" in long chat threads.
- **User Goal:** A clean, mobile-first dashboard that clearly displays a single "Total to Pay" number, making it easy to see their total debt across all friends in one place.

4. The Security-Conscious Peer

- **Profile:** Someone who has had disputes in the past where a friend claimed to have paid, but no record was kept.
- **Pain Point:** Disagreements over whether a debt was actually settled.
- **User Goal:** To utilize the **Two-Party Verification** system, ensuring a transaction only "disappears" once they personally click the "Received" button.

Technical Architecture

System Components

1. Frontend Layer (React & TypeScript)

- **React (Vite):** High-performance functional components for a modern, snappy UI.
- **TypeScript:** Strict typing for transaction objects and user states to prevent financial data errors.
- **Tailwind CSS:** Utility-first styling to achieve the minimalist, mobile-friendly "Glassmorphic" aesthetic.
- **Framer Motion:** Smooth animations for the "vanishing" request logic and layout transitions.
- **Zustand:** Lightweight state management to handle real-time balance updates across the dashboard.

2. Backend Layer (Spring Boot)

- **RESTful API:** Structured endpoints for request creation, friend management, and transaction history.
- **Spring Security & JWT:** Secure authentication flow to ensure users only access their own financial ledgers.
- **Spring Data JPA:** Efficient mapping of Java objects to the Supabase PostgreSQL schema.
- **Validation Layer:** Robust server-side checks to ensure amounts and friend tags are valid before processing.
- **Verification Middleware:** Custom logic to ensure transactions are only marked "Completed" when both the paid and received flags are true.

3. Settlement & Logic Engine

- **Algebraic Netting Module:** A dedicated service to calculate the sum of cross-debts between two users and generate a single consolidated balance.
- **Dual-Confirmation Workflow:** Logic gates that track the "Paid" (Payer) and "Received" (Payee) status, triggering the "archived" state only when both evaluate to true.
- **Auto-Metadata Handler:** Backend interceptors to automatically inject ISO timestamps and geolocation data into every new request.
- **State Machine:** Logic to handle the lifecycle of a request: Pending → Sender_Confirmed → Receiver_Confirmed → Resolved.

4. Storage & Infrastructure Layer (Supabase)

- **Database:** PostgreSQL (hosted on Supabase) for storing users, friendship relations, and transaction logs.
- **PostgreSQL Database:** Relational storage for complex ledger data, ensuring ACID compliance for financial integrity.
- **Supabase Auth:** Integrated OAuth and Email/Password management for secure user onboarding.
- **Real-time Subscriptions:** Supabase Realtime Webhook-based listeners that instantly update the UI when a friend marks a payment as "Paid" or "Received."

- **Row Level Security (RLS):** Database-level policies to ensure absolute data isolation between different friendship circles.

Data Models

1. User Profile

- Unique User ID (UUID)
- Display Name & Avatar URL
- Authentication Credentials (linked to Supabase Auth)
- Total Balance Summary (Calculated "To Pay" vs "To Receive")
- Notification Preferences

2. Payment Request (Transaction)

- Request ID & Title (e.g., "Dinner at Joe's")
- Amount (Decimal/BigInt)
- Creator ID (The user who initiated the request)
- Target Friend ID (The user tagged in the request)
- Verification Status:
 - `is_sender_confirmed` (Boolean: "I've Paid")
 - `is_receiver_confirmed` (Boolean: "I've Received")
- Lifecycle State (Pending, Partially Confirmed, Completed)
- Auto-generated Timestamp (Created_at & Updated_at)

3. Friendship Relation

- Connection ID
- User A ID & User B ID
- Status (Pending, Active, Blocked)
- Interaction Metadata (Last transaction date)
- Current Net Balance (Stored algebraic sum for quick dashboard rendering)

4. Settlement Summary (Netting Context)

- Grouped User ID pair
- List of active Request IDs included in the sum
- Aggregate "Pay" total vs Aggregate "Receive" total
- Calculated Net Difference (Algebraic Sum)
- Consensus Flags (Used when "Smart Settling" multiple requests at once)

Development Roadmap

Phase 1: Architecture Foundation & Scaffolding

- Project initialization with Spring Boot (Backend) and React/Vite (Frontend)
- Supabase integration for PostgreSQL database and Authentication
- Definition of core TypeScript interfaces and Java Data Models
- Implementation of secure JWT-based login and registration flow

Phase 2: Core Transaction Engine

- Development of the "Request" CRUD operations (Create, Read, Update, Delete)
- Implementation of the Dual-Confirmation state machine (Pending → Paid → Received)
- Backend logic for automatic metadata capture (Timestamps and User IDs)
- Basic mobile-responsive Dashboard displaying "Total to Pay" and "Total to Receive"
- Initial Friend Management system and tagging functionality

Phase 3: Smart Netting & Relationship Ledger

- Development of the Algebraic Sum algorithm to calculate net balances between users
- Implementation of the "Relationship Ledger" view with Current and Past transaction sections
- Integration of the "Vanishing" logic to archive settled requests automatically

- Real-time UI updates using Supabase database listeners to reflect payment status changes instantly
- Creation of the "Net Sum" toggle to consolidate multiple debts into a single settlement view

Phase 4: Aesthetic Polish & UX Optimization

- Implementation of the minimalist, "Glassmorphic" UI using Tailwind CSS
- Adding smooth animations with Framer Motion for transitioning settled items to history
- Fine-tuning the mobile-first navigation and thumb-zone interaction patterns
- Integration of push notification triggers for payment requests and confirmations
- Performance optimization for fast loading of transaction lists on mobile networks

Phase 5: Production Readiness & Deployment

- Security hardening using Supabase Row Level Security (RLS) and Spring Security
- Comprehensive Unit Testing for financial calculation logic and integration testing for APIs
- Setup of CI/CD pipelines via GitHub Actions for automated building and deployment
- Production deployment (Frontend on Vercel/Netlify, Backend on Render/AWS)
- Documentation of API endpoints and final UI/UX audit for consistency

Testing Strategy

1. Frontend Testing (React & TypeScript)

- **Unit Tests**
 - **Component Testing:** Validating UI elements like the "Algebraic Sum" toggle and Balance Cards using Vitest/React Testing Library.

- **Logic Tests:** Testing utility functions that format currency and calculate real-time UI totals.
- **State Management:** Testing Zustand store transitions for adding/removing requests from the active view.
- **Type Safety:** Comprehensive TypeScript checks to ensure data integrity across transaction interfaces.
- **Integration Tests**
 - **User Flow Integration:** Testing the interaction between the "Request Creator" modal and the Dashboard state.
 - **Supabase Client Tests:** Mocking real-time subscription listeners to ensure the UI reacts to database changes.
 - **API Layer Testing:** Verifying that frontend service functions correctly map to Spring Boot REST endpoints.

2. Backend Testing (Spring Boot)

- **Unit Tests**
 - **Service Layer Testing:** Using JUnit 5 and Mockito to validate the "Smart Netting" algebraic logic.
 - **State Machine Tests:** Verifying the transition logic for "Paid" and "Received" statuses to prevent premature archiving.
 - **Controller Tests:** Testing REST endpoints with `@WebMvcTest` to ensure correct HTTP response codes and JSON payloads.
 - **Validation Tests:** Ensuring the backend rejects invalid amounts (e.g., negative numbers) or unauthorized user IDs.
- **Integration Tests**
 - **Data Access Layer:** Testing Spring Data JPA repositories against a H2 in-memory database or a Supabase test instance.
 - **Security Integration:** Testing JWT filter chains to ensure protected routes (like the Ledger) are inaccessible without a valid token.
 - **Transactional Tests:** Verifying that "Netting" operations are atomic (either all requests archive or none do).

3. End-to-End (E2E) Testing

- **Critical Path Testing:** Using Playwright or Cypress to simulate two users: User A creating a request and User B marking it as paid.
- **Dual-Confirmation Simulation:** Automating the "Vanishing Logic" test where a request is only removed after both parties interact.
- **Mobile Responsiveness:** Testing touch interactions, swipe gestures, and UI scaling across various mobile device viewports.
- **Cross-Browser Testing:** Ensuring consistent rendering and functionality on Chrome, Safari, and Firefox.

4. Integration & Infrastructure Testing

- **Supabase Connectivity:** Testing the connection pool and Row Level Security (RLS) policies to ensure data isolation between friend groups.
- **Real-Time Sync Testing:** Validating that a "Received" click on one device instantly triggers the "Vanish" animation on the other device.
- **Container Health:** Using Docker Compose to run automated health checks on the Spring Boot container and database connectivity.
- **Load Testing:** Simulating a high volume of concurrent netting requests to ensure the Spring Boot logic remains performant under stress.

5. Deployment & Security Testing

- **API Security Audit:** Using automated tools to check for OWASP vulnerabilities in the Spring Boot REST API.
- **CI/CD Pipeline Validation:** Running the full test suite in GitHub Actions to prevent regressions before deployment.
- **Performance Monitoring:** Benchmarking the initial load time of the dashboard to ensure it remains under the 2-second threshold for mobile users.

Backend Structure

```
backend/
├── src/
│   └── main/
```

```

| | | — java/com/quali/
| | | | — EqualiApplication.java      # Main entry point
| | | | — config/                    # Configuration layer
| | | | | — SecurityConfig.java      # JWT and Supabase Auth
configuration
| | | | | — WebConfig.java           # CORS settings for mobile/web
| | | | | — SwaggerConfig.java       # API documentation (OpenAPI)
| | | | — controller/                # REST Endpoints
| | | | | — DashboardController.java # Home page summaries & top
friends
| | | | | — TransactionController.java # Request creation & status
updates
| | | | | — FriendController.java     # Friendship & Ledger management
| | | | | — NettingController.java    # Algebraic sum & batch settlement
| | | | — service/                   # Business Logic (The "Brain")
| | | | | — TransactionService.java   # Handles Dual-Confirmation logic
| | | | | — NettingService.java       # Algebraic sum calculation logic
| | | | | — NotificationService.java  # Real-time event triggers
| | | | — repository/               # Database Access (JPA/Supabase)
| | | | | — UserRepository.java
| | | | | — TransactionRepository.java
| | | | | — FriendshipRepository.java
| | | | — entity/                   # JPA Database Models
| | | | | — User.java
| | | | | — Transaction.java          # Paid/Received/Archived states
| | | | | — Friendship.java           # Maps relationships & net balances
| | | | — dto/                      # Data Transfer Objects (Request/Response)
| | | | | — TransactionRequestDTO.java
| | | | | — SettlementResponseDTO.java
| | | | | — DashboardSummaryDTO.java
| | | | — exception/                # Global Error Handling
| | | | | — GlobalExceptionHandler.java
| | | | | — ResourceNotFoundException.java
| | | — resources/
| | | | — application.properties    # Database & API keys
| | | | — db/migration/             # SQL scripts (Flyway/Liquibase)
| | — test/
| — java/com/quali/

```

			service/	# Logic & Netting tests
			controller/	# API integration tests
	— Dockerfile			# Containerization script
	— pom.xml / build.gradle			# Dependency management